



DOCUMENTACIÓN TÉCNICA

Sistema Avanzado de Gestión Académica Bellini

Unidad Educativa Particular Giovanni Bellini

Java 17 Spring Boot 3.3 React 18 + Vite PostgreSQL 15 JWT + RBAC

Documento: Manual Técnico
Versión: 2.0
Fecha: agosto de 2026
Basado en: Triviño & Cifuentes (2025) — Propuesta profesional SAGAB
Audiencia: equipo de desarrollo, soporte técnico y administración de infraestructura

1 Índice

1

Índice

2

Introducción y alcance

3

Arquitectura general

4

Estructura del repositorio

5

Modelo de datos

6

Seguridad

7

Backend — Spring Boot

8

Catálogo de API REST

9

Lógica de negocio destacada

10

Frontend — React

11

Variables de entorno

12

Instalación en entorno local

13

Despliegue en producción

14

Solución de problemas comunes

15

Mantenimiento y operación

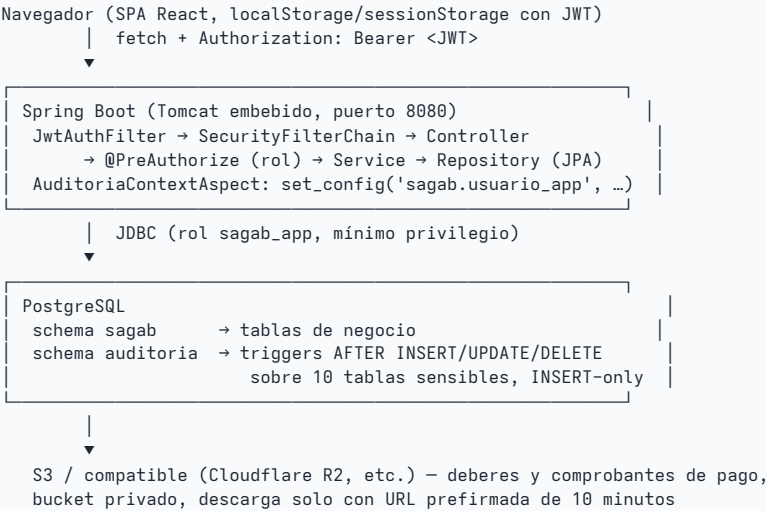
2 Introducción y alcance

SAGAB (Sistema Avanzado de Gestión Académica Bellini) es la plataforma de gestión académica, administrativa y financiera de la Unidad Educativa Particular Giovanni Bellini. Digitaliza los procesos de matrícula, calificaciones, asistencia, deberes, comunicación institucional, pagos (incluyendo transferencias bancarias con comprobante y revisión manual) y auditoría de cambios, con control de acceso basado en roles (RBAC) en dos capas: aplicación y base de datos.

El proyecto implementa la propuesta profesional de Triviño & Cifuentes (2025) y ha sido sometido a una auditoría técnica de seguridad interna (ver [INFORME_AUDITORIA.md](#) en la raíz del repositorio), cuyos hallazgos críticos y correcciones ya están incorporados al código (licenciamiento de librería PDF, cuentas demo, índices y validaciones de integridad financiera, particionamiento de auditoría, etc.).

3 Arquitectura general

CAPA	TECNOLOGÍA	RESPONSABILIDAD
Base de datos	PostgreSQL 15+	Esquema relacional (<code>sagab</code>), auditoría inmutable (<code>auditoria</code>), roles de conexión de mínimo privilegio, integridad referencial y de negocio (triggers, CHECK, índices parciales)
Backend	Java 17 · Spring Boot 3.3.5 (API REST)	Reglas de negocio, seguridad JWT/RBAC, generación de PDF (OpenPDF), envío de correo (SMTP), almacenamiento de archivos (S3)
Frontend	React 18 · TypeScript · Vite · Tailwind (shadcn/ui) · Bootstrap 5 (formularios escopados)	SPA consumida por administración, docentes, DECE, auditoría, representantes y estudiantes, con navegación condicionada por rol



4 Estructura del repositorio



5 Modelo de datos

Esquema `sagab` (negocio) + esquema `auditoria` (bitácora inmutable), PostgreSQL 15+. Construido incrementalmente por 18 scripts SQL numerados; Hibernate nunca modifica el esquema (`ddl-auto: validate`) — los scripts SQL son la única fuente de verdad.

5.1 Seguridad y RBAC de aplicación

TABLA	COLUMNAS CLAVE	NOTAS
usuario	email, username, hash_password, cedula, estado, intentos_fallidos, bloqueado_hasta, debe_cambiar_clave	1 fila por persona con acceso (admin, docente, representante, estudiante, DECE, auditor)

rol / permiso / rol_permiso	codigo, modulo	Matriz de permisos por rol (RBAC de aplicación, además del RBAC de Spring Security)
usuario_rol	id_usuario, id_rol	Relación N:M — un usuario puede tener más de un rol
refresh_token	token_hash (SHA-256), expira_en, revocado	Definida en el esquema; el flujo actual usa JWT de acceso de 30 min sin refresh activo

5.2 Estructura académica

TABLA	COLUMNAS CLAVE	NOTAS
periodo_academico	nombre, anio_lectivo, fecha_inicio/fin, activo	El período <code>activo=true</code> determina el año lectivo vigente para paralelos y rubros
paralelo	nivel, seccion, anio_lectivo, aforo_maximo	UNIQUE (nivel, seccion, anio_lectivo)
materia	codigo, nombre, area	Catálogo institucional
docente	id_usuario (1:1), titulo, fecha_ingreso	
representante	id_usuario (1:1), parentesco, direccion	Un representante puede tener varios <code>estudiante</code> a cargo
estudiante	codigo, cedula, id_paralelo, id_representante, id_usuario (1:1)	Datos de menores — LOPDP. <code>id_usuario</code> es la cuenta del Portal Familiar, creada automáticamente en la matrícula
asignacion_docente	id_docente, id_materia, id_paralelo, id_periodo	UNIQUE (materia, paralelo, período) — un solo docente por combinación

5.3 Académico — calificaciones y asistencia

TABLA	COLUMNAS CLAVE	NOTAS
calificacion	parcial (1–3), nota_tarea/clase/examen, promedio (columna generada)	$\text{promedio} = \text{tarea} \times 0.20 + \text{clase} \times 0.20 + \text{examen} \times 0.60$, calculado por PostgreSQL, no por la app. CHECK 1–10. UNIQUE (estudiante, asignación, parcial)
asistencia	estado (enum), justificacion, documento_url	UNIQUE (estudiante, fecha) — un registro por estudiante por día
incidencia_disciplinaria	gravedad, estado, acta_url, firmada_por_representante	Tabla creada en el esquema; el módulo disciplinario (controller/service) está pendiente de implementar

5.4 Financiero

TABLA	COLUMNAS CLAVE	NOTAS
rubro	tipo (MATRICULA/PENSION/EXTRACURRICULAR/OTRO), nombre, valor, anio_lectivo	Motivos de pago del año lectivo vigente — poblados por <code>GET /api/finanzas/rubros</code>
obligacion_pago	id_estudiante, id_rubro, mes, valor, fecha_vencimiento, estado	UNIQUE (estudiante, rubro, mes). Se genera manualmente (admin) o automáticamente cuando la familia paga un rubro que aún no tenía obligación (ver §9.3)
pago	valor_pagado, metodo, numero_recibo, banco_origen, numero_referencia, asunto, comprobante_url/hash/mime/tamaño, estado_revision, revisado_por, observaciones_admin	EFFECTIVO/CHEQUE/TARJETA nacen APROBADO ; TRANSFERENCIA nace EN_REVISION hasta que un admin la aprueba/rechaza

Trigger `fn_validar_suma_pagos` (BEFORE INSERT/UPDATE en pago): impide que la suma de pagos **APROBADOS** de una obligación supere su valor. Los pagos **EN_REVISION** o **RECHAZADO** no cuentan para el total hasta que se aprueban.

5.5 Mensajería, notificaciones y deberes

TABLA	COLUMNAS CLAVE	NOTAS
<code>mensaje / mensaje_destinatario</code>	<code>asunto</code> , <code>cuerpo</code> , <code>es_circular</code> , <code>leido_en</code>	Mensajería interna con confirmación de lectura
<code>notificacion</code>	<code>id_destinatario</code> , <code>materia</code> , <code>calificacion</code> , <code>mensaje</code> , <code>leido_en</code>	Se dispara automáticamente cuando un estudiante saca nota < 7/10 (estudiante y representante)
<code>tarea</code>	<code>id_asignacion</code> , <code>titulo</code> , <code>descripcion</code> , <code>fecha_limite</code>	Publicada por el docente
<code>entrega_tarea</code>	<code>estado</code> (PENDIENTE/ENTREGADO/REVISADO), <code>archivo_url/nombre/mime/tamaño</code> , <code>observacion_docente</code>	Se genera una fila por cada estudiante activo del paralelo al crear la tarea. UNIQUE (tarea, estudiante)

5.6 Auditoría (esquema auditoria)

OBJETO	DESCRIPCIÓN
<code>registro_cambio</code>	Particionada por mes. <code>datos_antes / datos_despues</code> en JSONB, <code>columnas_modificadas</code> , <code>usuario_app</code> , <code>ip_cliente</code> . El hash de contraseña se elimina del JSONB antes de guardar.
<code>evento_seguridad</code>	LOGIN, LOGIN_FALLIDO, LOGOUT, ACCESO_DENEGADO, EXPORTACION
Triggers <code>trg_audit_*</code>	<code>AFTER INSERT/UPDATE/DELETE</code> sobre 10 tablas: <code>usuario</code> , <code>estudiante</code> , <code>calificacion</code> , <code>asistencia</code> , <code>incidencia_disciplinaria</code> , <code>obligacion_pago</code> , <code>pago</code> , <code>tarea</code> , <code>entrega_tarea</code> (+ las que se sumen)
Triggers <code>trg_inmutable_*</code>	<code>BEFORE UPDATE/DELETE</code> — bloquean cualquier alteración posterior de la bitácora, incluso por el superusuario de aplicación
<code>fn_crear_particion_siguiente_mes()</code>	Crea la partición mensual siguiente; se invoca automáticamente por <code>AuditoriaParticionScheduler</code> (backend) además de poder programarse por <code>cron/pg_partman</code>

5.7 Roles de conexión PostgreSQL (defensa en profundidad)

ROL	PRIVILEGIOS	USO
<code>sagab_app</code>	CRUD en <code>sagab</code> , solo INSERT/SELECT en <code>auditoria</code> . Sin DDL.	Conexión del backend Spring Boot
<code>sagab_auditor</code>	Solo lectura de <code>auditoria</code> + catálogos mínimos	Consulta externa de auditores
<code>sagab_readonly</code>	Solo lectura de <code>sagab</code>	Reportería / BI

Ningún rol de conexión tiene privilegios DDL (`REVOKE CREATE ON SCHEMA sagab, auditoria FROM PUBLIC`) — reduce drásticamente el impacto de una eventual inyección SQL o credencial filtrada.

6 Seguridad

6.1 Autenticación y sesión

- **JWT HS256**, expiración de 30 minutos, roles embebidos en el token. API completamente *stateless* (sin sesión de servidor).
- **Login por username** (no por email) — `AuthService.login()`. El JWT se guarda en `sessionStorage` del navegador (se pierde al cerrar la pestaña).
- **BCrypt fuerza 12** para contraseñas. El hash nunca sale en DTOs ni queda en la auditoría (el trigger lo elimina del JSONB antes de persistir).
- **Bloqueo por fuerza bruta**: 5 intentos fallidos → cuenta bloqueada 15 minutos. Todo intento (éxito o fallo) se registra en `auditoria.evento_seguridad`.

- **Cambio de contraseña forzado** (`debe_cambiar_clave=true`): se activa automáticamente para toda cuenta nueva (estudiante en matrícula, representante nuevo) — el frontend intercepta este estado y bloquea la navegación hasta completar `CambiarClaveScreen`.

6.2 Autorización

- **Dos niveles:** por ruta (`SecurityFilterChain` en `SecurityConfig`) y por método (`@PreAuthorize` en cada controlador) — defensa en profundidad, un fallo en un nivel no compromete el sistema.
- **Roles:** `ADMIN`, `DOCENTE`, `REPRESENTANTE`, `ESTUDIANTE`, `DECE`, `AUDITOR`.
- **Regla "es propio":** `EstudianteService.esPropio(idEstudiante, auth)` es el punto único que decide si un representante/estudiante puede ver o modificar datos de un estudiante en concreto. La usan Calificaciones, Asistencia, Finanzas y Tareas — evita duplicar esta lógica de seguridad en cada módulo.

6.3 Cabeceras y transporte

- **CORS** restringido a los orígenes configurados en `SAGAB_CORS` (lista, no wildcard).
- **HSTS** (`includeSubDomains`, 1 año) y `X-Frame-Options: DENY`.
- **CSRF deshabilitado** deliberadamente: la API es stateless con JWT en cabecera `Authorization`, no en cookies, por lo que CSRF no aplica.

6.4 Manejo de errores y fuga de información

- `server.error.include-stacktrace: never` — nunca se filtran detalles internos al cliente.
- Mensajes de error genéricos en login: no se revela si un usuario/cédula existe o no.
- `SecretsGuard` (`config/SecretsGuard.java`): impide que la aplicación arranque si `SAGAB_DB_PASSWORD` o `SAGAB_JWT_SECRET` no están definidas explícitamente como variable de entorno — evita el fallback silencioso a los valores de ejemplo del repositorio.

6.5 Validación de archivos subidos

`FileValidationService` valida deberes y comprobantes de pago sin confiar en el `Content-Type` ni la extensión declarados por el cliente (ambos falsificables):

- Tamaño máximo configurable por tipo de subida.
- **Firma binaria (magic bytes):** solo se acepta contenido cuyos primeros bytes correspondan de verdad a PDF, JPEG, PNG, WEBP, ZIP/DOCX o DOC antiguo (OLE).
- **Hash SHA-256** del contenido — detecta comprobantes de pago duplicados subidos dos veces para la misma obligación.

6.6 Almacenamiento de archivos

Bucket S3 (o compatible: Cloudflare R2, etc.) siempre privado. `StorageService` nunca expone el bucket como público: toda descarga se hace con una **URL prefirmada de 10 minutos**, generada solo después de que el backend confirmó que el usuario autenticado tiene permiso sobre ese archivo en concreto (misma regla de autorización que el resto de la API). Si las variables `SAGAB_S3_*` no están configuradas, los endpoints de subida devuelven un error controlado en vez de un 500 genérico (`StorageService.isConfigured()`).

7 Backend — Spring Boot

7.1 Dependencias clave (pom.xml)

DEPENDENCIA	USO
spring-boot-starter-web / data-jpa / security / validation / aop / mail	API REST, ORM (Hibernate), seguridad, validación de DTOs, aspectos (contexto de auditoría), envío de correo SMTP
org.postgresql:postgresql	Driver JDBC

io.jsonwebtoken (jjwt-api/impl/jackson) 0.12.6	Emisión y verificación de JWT HS256
com.github.librepdf:openpdf 1.3.30	Generación de boletines/reportes PDF. Se eligió sobre iText7 (AGPL v3) porque este último obligaría a liberar el código fuente del backend al prestarlo por red sin licencia comercial — ver <code>INFORME_AUDITORIA.md</code> , hallazgo crítico #6
software.amazon.awssdk:s3 2.51.0	Almacenamiento de archivos compatible S3
lombok	Reduce boilerplate en entidades (<code>@Getter</code> / <code>@Setter</code>)

7.2 Patrón de capas

Todo módulo sigue estrictamente `entity → repository → service → controller → dto`:

- **model** — entidades JPA anotadas, sin lógica de negocio.
- **repository** — interfaces Spring Data JPA; consultas derivadas por nombre de método o `@Query` nativa/JPQL cuando hace falta (búsqueda por trigram, agregaciones).
- **service** — lógica de negocio, transacciones (`@Transactional`), autorización de datos ("es propio"), orquestación de repositorios.
- **controller** — solo enrutamiento HTTP y `@PreAuthorize` ; delega todo al service.
- **dto** — records de Java, uno por request/response; nunca se exponen las entidades JPA directamente (evita fugas de campos internos como `hash_password`).

7.3 Entidades (19)

Usuario · Rol · Docente · Representante · Estudiante ·
PeriodoAcademico · Paralelo · Materia · AsignacionDocente ·
Calificacion · Asistencia · Rubro · ObligacionPago · Pago ·

Mensaje · MensajeDestinatario · Notificacion · Tarea ·
EntregaTarea

7.4 Servicios (18)

SERVICIO	RESPONSABILIDAD
AuthService	Login, bloqueo por fuerza bruta, cambio de contraseña
JwtService	Emisión/verificación de JWT
EventoSeguridadService	Registro de eventos de seguridad
MatriculaService	Alta de estudiante + creación automática de cuentas (estudiante y representante)
EstudianteService	Búsqueda, "mis estudiantes", regla <code>esPropio</code>
CalificacionService	Registro masivo de notas, disparo de notificación si nota < 7
AsistenciaService	Registro diario, ausencias consecutivas (alerta DECE)
AsignacionDocenteService	Asignaciones del docente autenticado
ParaleloService	Paralelos del año lectivo vigente
FinanzasService	Estado de cuenta, registro directo de pago, subida de comprobante, cola de revisión, aprobar/rechazar, rubros disponibles
TareaService	Publicar deber, entregas, revisión docente, descarga
MensajeService	Bandeja de entrada, marcar leído
NotificacionService	Notificaciones del usuario, marcar leída
DashboardService	KPIs de la pantalla de Inicio (no representante)
StorageService	Subida/descarga S3, URLs prefiradas
FileValidationService	Validación de archivos (magic bytes, tamaño, hash)

8 Catálogo de API REST

Todas las rutas bajo `/api`. Autenticación: cabecera `Authorization: Bearer <JWT>`, excepto `POST /api/auth/login`.

8.1 Autenticación

MÉTODO	RUTA	ROLES	DESCRIPCIÓN
POST	<code>/api/auth/login</code>	público	Login por username; devuelve JWT, roles, nombre y <code>debeCambiarClave</code>
POST	<code>/api/auth/cambiar-clave</code>	autenticado	Cambia la contraseña propia

8.2 Matrícula y estudiantes

MÉTODO	RUTA	ROLES	DESCRIPCIÓN
POST	<code>/api/matriculas</code>	ADMIN	Alta de estudiante; crea automáticamente su cuenta (y la del representante si es nuevo)
GET	<code>/api/estudiantes/paralelo/{id}</code>	DOCENTE, ADMIN	Nómina de un paralelo
GET	<code>/api/estudiantes/mios</code>	REPRESENTANTE, ESTUDIANTE	Estudiantes propios (Portal Familiar)
GET	<code>/api/estudiantes/buscar?q=</code>	ADMIN	Búsqueda por nombre, cédula del estudiante o cédula del representante (<code>pg_trgm</code> + match exacto de cédula)
GET	<code>/api/paralelos</code>	ADMIN	Paralelos del año lectivo vigente

8.3 Académico

MÉTODO	RUTA	ROLES	DESCRIPCIÓN
POST	<code>/api/calificaciones</code>	DOCENTE, ADMIN	Ingreso/edición masiva de notas
GET	<code>/api/calificaciones/asignacion/{id}/parcial/{p}</code>	DOCENTE, ADMIN	Notas por asignación y parcial
GET	<code>/api/calificaciones/estudiante/{id}</code>	DOCENTE, ADMIN, REPRESENTANTE, ESTUDIANTE	Notas de un estudiante en todas sus materias
GET	<code>/api/calificaciones/buscar</code>	DOCENTE, ADMIN	Búsqueda avanzada (estudiante/curso/materia/período/docente/parcial)
DELETE	<code>/api/calificaciones/{id}</code>	DOCENTE (propias), ADMIN	Elimina una calificación
GET	<code>/api/asignaciones/mias</code>	DOCENTE, ADMIN	Asignaciones del docente autenticado

8.4 Asistencia

MÉTODO	RUTA	ROLES	DESCRIPCIÓN
POST	<code>/api/asistencia</code>	DOCENTE, ADMIN	Registro diario del paralelo + disparo de alertas DECE
GET	<code>/api/asistencia/paralelo/{id}</code>	DOCENTE, ADMIN	Registro del paralelo (por fecha opcional)
GET	<code>/api/asistencia/paralelo/{id}/consecutivas</code>	DOCENTE, ADMIN	Ausencias injustificadas consecutivas por estudiante
GET	<code>/api/asistencia/estudiante/{id}</code>	DOCENTE, ADMIN, REPRESENTANTE, ESTUDIANTE	Historial (últimos 6 meses por defecto)

8.5 Deberes

MÉTODO	ruta	ROLES	DESCRIPCIÓN
POST	/api/tareas	DOCENTE	Publicar un deber (genera una entrega PENDIENTE por estudiante activo del paralelo)
GET	/api/tareas/asignacion/{id}	DOCENTE, ADMIN	Deberes de una asignación
GET	/api/tareas/{id}/entregas	DOCENTE, ADMIN	Entregas de todos los estudiantes de una tarea
GET	/api/tareas/estudiante/{id}	DOCENTE, ADMIN, REPRESENTANTE, ESTUDIANTE	Entregas de un estudiante (Portal Familiar)
POST	/api/tareas/{id}/estudiante/{id}/entrega	REPRESENTANTE, ESTUDIANTE	Sube el archivo de la entrega (multipart)
POST	/api/tareas/entregas/{id}/revisar	DOCENTE	Califica/comenta una entrega
GET	/api/tareas/entregas/{id}/descarga	DOCENTE, ADMIN, REPRESENTANTE, ESTUDIANTE	URL prefirmada de descarga

8.6 Financiero

MÉTODO	ruta	ROLES	DESCRIPCIÓN
GET	/api/finanzas/estudiante/{id}	ADMIN, REPRESENTANTE, ESTUDIANTE	Estado de cuenta del estudiante
GET	/api/finanzas/rubros	ADMIN, REPRESENTANTE, ESTUDIANTE	Motivos de pago disponibles (rubros del año lectivo vigente)
POST	/api/finanzas/pagos	ADMIN	Registro directo (efectivo/cheque/tarjeta) — nace APROBADO
POST	/api/finanzas/pagos/transferencia	REPRESENTANTE, ESTUDIANTE	Sube comprobante de transferencia (multipart). Acepta <code>idObligacion</code> (obligación existente) o <code>idRubro + idEstudiante</code> (motivo directo — genera la obligación del mes automáticamente). Nace EN_REVISION
GET	/api/finanzas/pagos/revision	ADMIN	Cola de comprobantes EN_REVISION
POST	/api/finanzas/pagos/{id}/aprobar	ADMIN	Aprueba el pago; actualiza el estado de la obligación si corresponde
POST	/api/finanzas/pagos/{id}/rechazar	ADMIN	Rechaza el pago (con observación opcional)
GET	/api/finanzas/pagos/{id}/comprobante	ADMIN, REPRESENTANTE, ESTUDIANTE	URL prefirmada del comprobante

8.7 Mensajería y notificaciones

MÉTODO	ruta	ROLES	DESCRIPCIÓN
GET	/api/mensajes/mias	autenticado	Bandeja de entrada
POST	/api/mensajes/{id}/leido	autenticado	Marcar como leído
GET	/api/notificaciones/mias	autenticado	Notificaciones propias
POST	/api/notificaciones/{id}/leida	autenticado	Marcar como leída

8.8 Dashboard y auditoría

MÉTODO	ruta	ROLES	DESCRIPCIÓN
GET	/api/dashboard/resumen	ADMIN, DOCENTE, AUDITOR, DECE	KPIs de la pantalla de Inicio
GET	/api/auditoria/cambios	AUDITOR, ADMIN	Historial de cambios (filtros por tabla/usuario, paginado)
GET	/api/auditoria/eventos	AUDITOR, ADMIN	Eventos de seguridad (logins, fallos, exportaciones)
GET	/api/auditoria/historial/{tabla}/{idFila}	AUDITOR, ADMIN	Trazabilidad completa de una fila específica

9 Lógica de negocio destacada

9.1 Creación automática de cuentas en la matrícula

`MatriculaService.crear()` hace, en una sola transacción:

1. Valida cédula del estudiante y del representante (algoritmo módulo 10 ecuatoriano), y que no sean iguales entre sí.
2. Si el correo del representante ya existe con rol REPRESENTANTE, reutiliza esa cuenta (un representante puede tener varios hijos). Si existe con otro rol, **rechaza explícitamente** — evita escalamiento de privilegios silencioso.
3. Si el representante es nuevo: genera username (`UsernameGenerator` , patrón `nombre.apellido` con desambiguación numérica si ya existe) y una **contraseña temporal aleatoria** (alfabeto sin caracteres ambiguos), hasheada con BCrypt(12). Se devuelve **una sola vez** en la respuesta al admin.
4. Crea siempre la cuenta del **estudiante**: username igual al patrón anterior, contraseña inicial = su propia cédula (hasheada con BCrypt). `debe_cambiar_clave=true` en ambos casos — el frontend fuerza `CambiarClaveScreen` en el primer ingreso.
5. Genera el código institucional (`EST-0001` , secuencia `estudiante_codigo_seq`) y persiste el estudiante enlazado a paralelo, representante y su propia cuenta de usuario.

9.2 Cálculo de promedios

El promedio **no lo calcula la aplicación**: es una columna generada de PostgreSQL (`calificacion.promedio`), garantizando que nunca pueda quedar desincronizado de las notas parciales: `ROUND(tarea×0.20 + clase×0.20 + examen×0.60, 2)` . Umbral de riesgo académico: promedio < 7/10.

9.3 Flujo financiero — pago por transferencia sin obligación previa

Para evitar que un administrador tenga que crear a mano una `obligacion_pago` por cada estudiante antes de que puedan pagar, `FinanzasController.subirComprobante()` admite dos formas de indicar qué se paga:

- **idObligacion**: una obligación ya existente (creada por un admin, o por una subida anterior para ese mismo mes).
- **idRubro + idEstudiante**: el "motivo de pago" directamente. `FinanzasService.obtenerOCrearObligacionDelMes()` busca una obligación para (estudiante, rubro, primer día del mes en curso); si no existe, la crea con el `valor` del rubro. El motivo especial **"Otro (especifique)"** (tipo `OTRO` , valor base \$0) permite monto libre: si `valorPagado` supera el valor de la obligación resuelta, esta se ajusta automáticamente hacia arriba antes de crear el pago, para no chocar con el trigger de suma de pagos.

Aprobar un pago dispara `actualizarEstadoSiCorresponde()` : si la suma de pagos APROBADOS de la obligación alcanza su valor, la obligación pasa a `PAGADO` .

9.4 Alertas automáticas

EVENTO	DISPARADOR	EFEECTO
Nota < 7/10	<code>CalificacionService</code> al guardar	Crea <code>notificacion</code> para el estudiante (si tiene cuenta) y su representante

Ausencias injustificadas consecutivas	<code>AsistenciaService.consecutivasPorParalelo()</code>	Se expone al docente/admin como alerta en la tabla de registro (seguimiento DECE)
5 intentos de login fallidos	<code>AuthService</code>	Bloqueo de cuenta 15 minutos + registro en <code>evento_seguridad</code>

10 Frontend — React

10.1 Estructura

ARCHIVO/CARPETA	CONTENIDO
<code>src/api/client.ts</code>	Cliente fetch central: JWT desde <code>sessionStorage</code> , manejo uniforme de 401 y errores (<code>ApiError</code>)
<code>src/api/auth.ts</code>	<code>login()</code> / <code>logout()</code> / <code>cambiarClave()</code>
<code>src/api/sagab.ts</code>	Servicios tipados por módulo: matriculas, estudiantes, calificaciones, asistencia, tareas, finanzas, mensajes, notificaciones, paralelos
<code>src/app/App.tsx</code>	Enrutamiento por rol, <code>lazy()</code> por vista (cada pantalla es su propio chunk)

10.2 Navegación por rol

ROL	MÓDULOS VISIBLES (SIDEBAR)
ADMIN	Inicio, Matrícula, Académico, Asistencia, Deberes, Financiero, Portal Familiar (vista previa)
DOCENTE	Inicio, Académico, Asistencia, Deberes
DECE	Inicio, Asistencia
AUDITOR	Inicio
REPRESENTANTE / ESTUDIANTE	Portal Familiar (layout propio, sin Sidebar administrativo)

Mapecto en `NAV_POR_ROL` (`components/Sidebar.tsx`). El estudiante y su representante comparten exactamente las mismas capacidades dentro del Portal Familiar (entregar deberes, pagar) — la diferencia es solo de qué estudiante(s) ven.

10.3 Portal Familiar — módulo de Pagos

Pestaña "Pagos" del Portal Familiar, componentes en `ParentPortal.tsx` :

- `NuevoPagoForm` : siempre visible. Formulario vertical con validación en tiempo real (clases Bootstrap `is-valid` / `is-invalid` según `touched`). Selector "Motivo de pago" poblado desde `GET /api/finanzas/rubros` ; si se elige "Otro (especifique)" aparecen campos adicionales de motivo libre y monto editable.
- `ObligacionCard` : una tarjeta por cada obligación ya existente del estudiante, con su propio formulario de subida de comprobante (mismos campos, `idObligacion` fijo).

11 Variables de entorno

VARIABLE	PROPÓSITO	NOTAS
<code>SAGAB_DB_HOST/PORT/NAME</code>	Conexión PostgreSQL	Default: localhost/5432/sagab
<code>SAGAB_DB_USER/PASSWORD</code>	Credenciales del rol de aplicación	Obligatoria (SecretsGuard); local dev usa el placeholder de <code>04_rols_bd.sql</code>
<code>SAGAB_JWT_SECRET</code>	Clave HMAC-SHA256 ≥32 bytes	Obligatoria (SecretsGuard); generar con <code>openssl rand -base64 48</code> , distinta por entorno

SAGAB_SMTP_HOST/USER/PASS	Envío de correo	Default host: smtp.gmail.com
SAGAB_CORS	Orígenes permitidos (lista)	Default: http://localhost:5173
SAGAB_S3_ENDPOINT/REGION/BUCKET/ACCESS_KEY/SECRET_KEY	Almacenamiento de archivos	Si vacías, la app arranca igual; los endpoints de subida devuelven error controlado
SAGAB_UPLOAD_MAX_MB_DEBERES / _COMPROBANTE	Tamaño máximo por tipo de archivo	Default: 15MB / 8MB
PORT	Puerto HTTP	Default: 8080
VITE_API_URL (frontend)	URL base del backend	frontend/.env , default http://localhost:8080

12 Instalación en entorno local

12.1 Base de datos

```
createdb sagab
psql -d sagab -f database/01_schema.sql
psql -d sagab -f database/02_indices.sql
psql -d sagab -f database/03_auditoria.sql
psql -d sagab -f database/04_rols_bd.sql
psql -d sagab -f database/06_matricula_campos.sql
psql -d sagab -f database/07_usuario_username.sql
psql -d sagab -f database/09_estudiante_usuario.sql
psql -d sagab -f database/10_asignacion_saida_historia.sql
psql -d sagab -f database/11_fix_usernames_demo.sql
psql -d sagab -f database/12_indice_check_pagos.sql
psql -d sagab -f database/13_particion_auditoria_automatizada.sql
psql -d sagab -f database/14_notificaciones.sql
psql -d sagab -f database/15_deberes.sql
psql -d sagab -f database/16_pagos_transferencia.sql
psql -d sagab -f database/17_pago_asunto.sql
psql -d sagab -f database/18_rubros_motivos_pago.sql

# Solo para desarrollo (datos ficticios, NO producción):
psql -d sagab -f database/05_datos_prueba.sql
psql -d sagab -f database/08_usuario_aaron.sql
```

database/setup_db.sh automatiza los pasos base, pero no incluye todavía todos los scripts incrementales (12 en adelante) — ejecútalos a mano en orden como arriba, o actualiza el script.

12.2 Backend

```
cd backend
export SAGAB_DB_PASSWORD='CAMBIAR_EN_PRODUCCION_app' # o la que hayas puesto en 04_rols_bd.sql
export SAGAB_JWT_SECRET="$(openssl rand -base64 48)"
mvn spring-boot:run # http://localhost:8080
```

12.3 Frontend

```
cd frontend
cp .env.example .env # ajustar VITE_API_URL si aplica
pnpm install # o npm install
pnpm dev # http://localhost:5173
```

12.4 Generar el hash BCrypt de un usuario semilla

```
System.out.println(new BCryptPasswordEncoder(12).encode("TuContraseñaSegura123"));
```

13 Despliegue en producción

13.1 Render (backend + PostgreSQL) — render.yaml

Blueprint incluido: crea una base PostgreSQL administrada (`sagab-db`) y un servicio web Docker (`backend/Dockerfile` , build multi-stage Maven → JRE Alpine). Las variables de conexión a la base se inyectan automáticamente desde el recurso de base de datos; `SAGAB_JWT_SECRET` se genera automáticamente por Render (`generateValue: true`). Ajustar `SAGAB_CORS` al dominio real del frontend desplegado.

13.2 Servidor local institucional (alternativa)

- 1. HTTPS obligatorio: certificado TLS (Let's Encrypt o interno) en un reverse proxy Nginx delante de Tomcat.
- 2. Cambiar todas las credenciales marcadas `CAMBIAR_EN_PRODUCCION` y definir los secretos por variables de entorno reales.
- 3. Respaldo diario: `pg_dump -Fc sagab` vía cron hacia un servidor secundario.
- 4. Crear la partición mensual de auditoría (cron o `pg-partman`) — o confirmar que `AuditoriaParticionScheduler` esté activo.
- 5. Restringir `pg_hba.conf` a la red institucional y desactivar el acceso remoto al superusuario.

14 Solución de problemas comunes

SÍNTOMA	CAUSA HABITUAL	SOLUCIÓN
FATAL: la autenticación password falló para el usuario "sagab_app"	El rol de Postgres ya existía con otra contraseña, o nunca se corrió <code>04_roles_bd.sql</code>	<code>ALTER ROLE sagab_app WITH PASSWORD '...'</code> para que coincida con <code>SAGAB_DB_PASSWORD</code> ; revisar variables de entorno persistentes del sistema
<code>SAGAB_JWT_SECRET</code> no está definida al arrancar	<code>SecretsGuard</code> exige la variable de entorno explícita, no basta con el default de <code>application.yml</code>	Exportar <code>SAGAB_JWT_SECRET</code> antes de <code>mvn spring-boot:run</code> (ver §12.2)
Un endpoint nuevo devuelve 404 aunque el código ya lo tiene	Spring Boot no hace hot-reload de nuevas rutas sin <code>spring-boot-devtools</code> (no incluido en este proyecto)	Detener el proceso (<code>kill</code> o Ctrl+C) y volver a <code>mvn spring-boot:run</code>
Un <code><select></code> que depende de datos de catálogo aparece vacío	Falta ejecutar la migración SQL que siembra esos datos (p. ej. <code>18_rubros_motivos_pago.sql</code>)	Verificar con <code>SELECT * FROM sagab.<tabla></code> y correr la migración pendiente
401/403 inesperado tras cambios de rol	El JWT ya emitido lleva los roles "congelados" del momento del login	Cerrar sesión y volver a iniciar para obtener un token con los roles actualizados
Error genérico al subir un comprobante/deber	Archivo no coincide con las firmas binarias permitidas, o excede el tamaño máximo	Revisar <code>FileValidationService</code> y las variables <code>SAGAB_UPLOAD_MAX_MB_*</code>

15 Mantenimiento y operación

- **Particiones de auditoría:** `AuditoriaParticionScheduler` corre en el backend; verificar en logs "Verificación de partición de auditoría completada" en cada arranque. Como respaldo, `pg-partman` o un cron externo.
- **Rotación de secretos:** `SAGAB_JWT_SECRET` , credenciales SMTP y credenciales S3 deben rotarse periódicamente. Rotar el JWT secret invalida todas las sesiones activas (esperado).
- **Ajuste de rubros:** los valores de Matrícula/Mensualidad/Uniforme sembrados por `18_rubros_motivos_pago.sql` son placeholders — actualizarlos con `UPDATE sagab.rubro SET valor = ... WHERE nombre = '...'` a los montos reales de la

institución.

- **Monitoreo:** revisar periódicamente `auditoria.evento_seguridad` para detectar patrones de fuerza bruta o accesos anómalos.
- **Módulos pendientes** (mismo patrón entity/repo/service/controller que el resto): disciplinario (tabla `incidencia_disciplinaria` ya creada), boletines PDF con OpenPDF, envío de notificaciones por correo con `spring-boot-starter-mail` (dependencia ya configurada).

Ver `INFORME_AUDITORIA.md` en la raíz del repositorio para el detalle completo de la auditoría técnica de seguridad realizada sobre el proyecto.